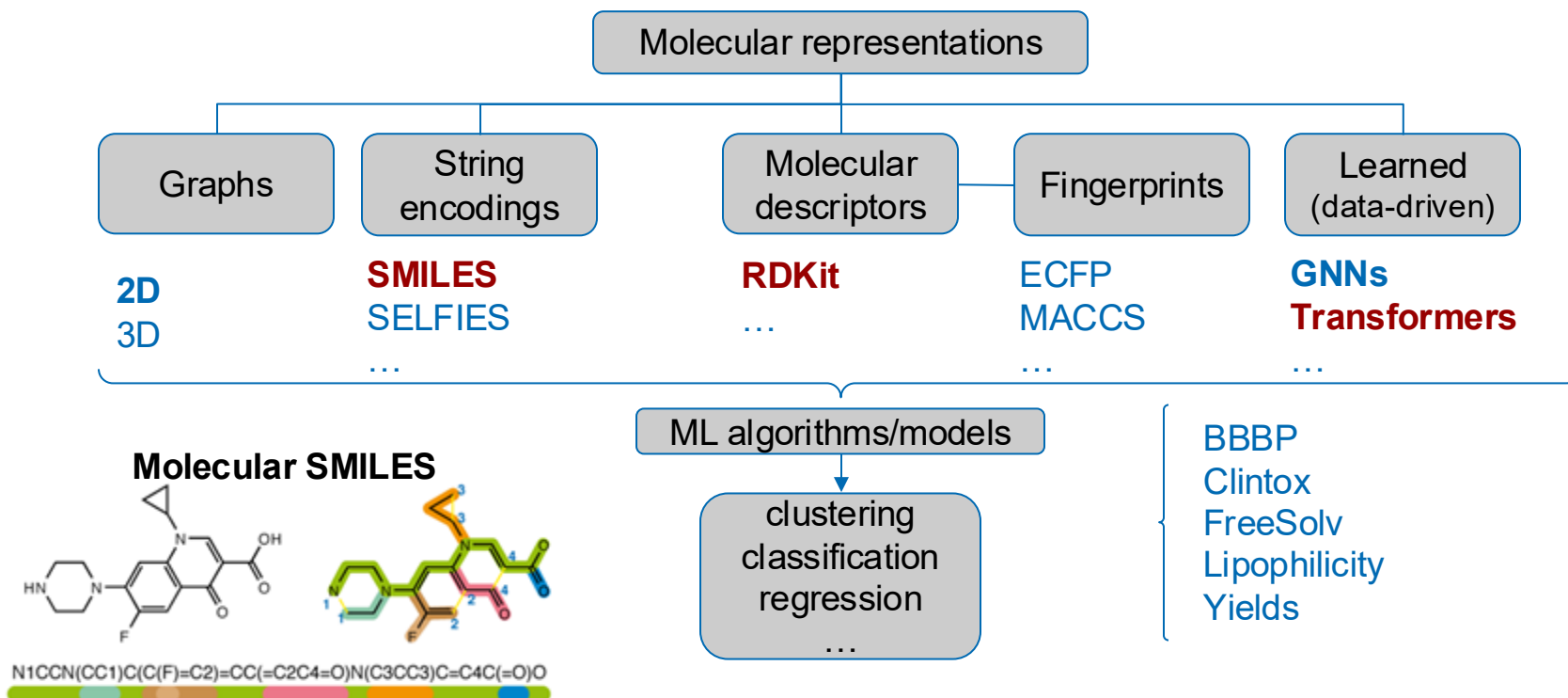


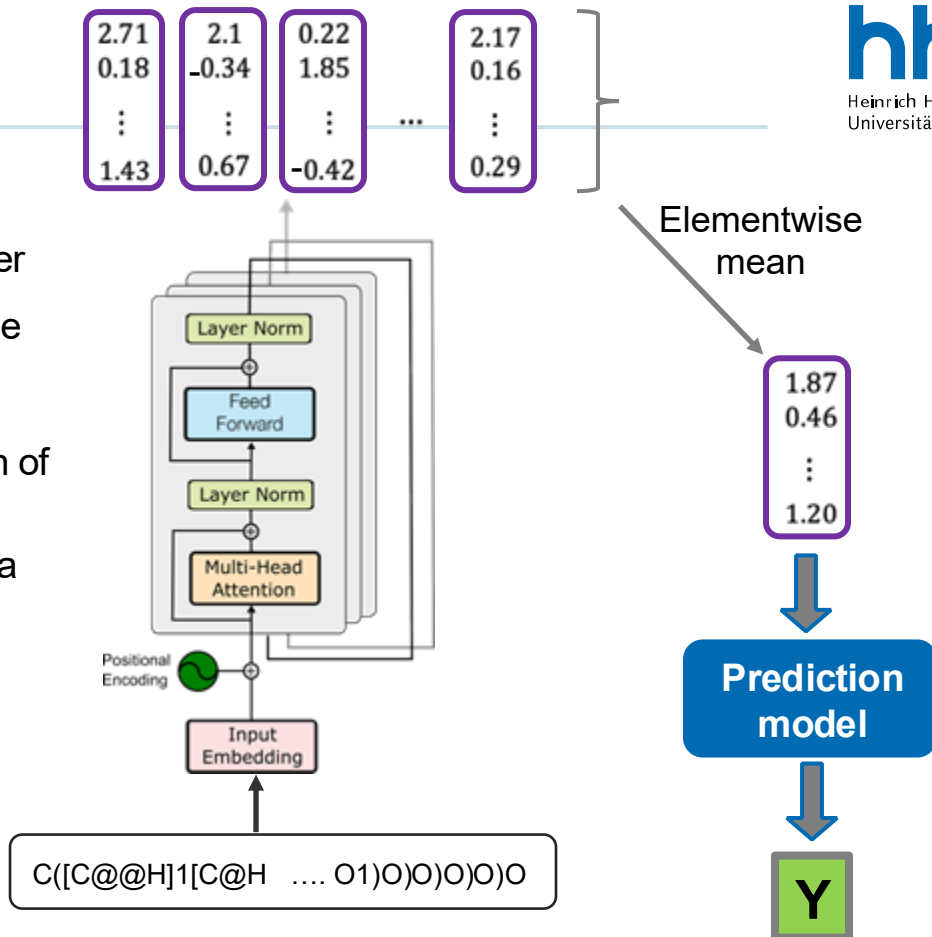
Transformer Networks for small molecules

Molecular numerical representations



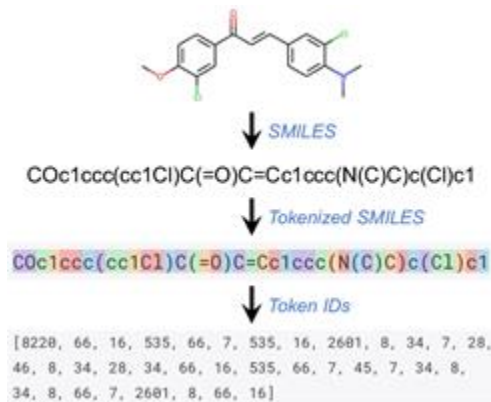
Motivation

- We want to use SMILES strings as the input for a Transformer Network Encoder
- After training, we want to extract a single numerical vector representing the small molecule
 - For example, the elementwise mean of all token representations
- Use the resulting vector as the input of a machine learning prediction model



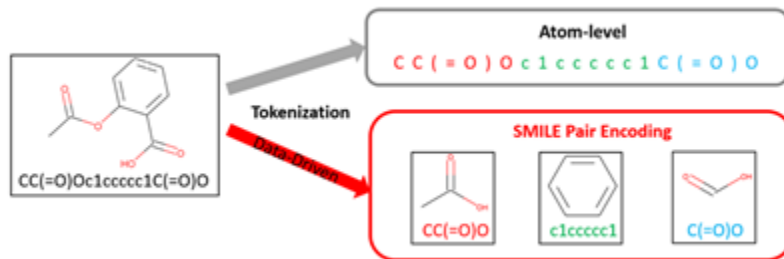
Tokenization of SMILES strings

- Not as straightforward as for protein amino acid sequences.
- Tokenization as a spectrum:
 - **Atomic** (character) level
Lossless representation but **inefficient** (long tokenized sequences and overlooks meaningful semantic structures).
 - **Molecular** (word) level
 Intuitive representation, but struggles with **out-of-vocabulary** molecules during inference.
 - **Sub-molecular** (sub-word) level
 Manages rare and unseen words while maintaining computational efficiency.
 - Search for most **common patterns** (SMILES pair encoding)
 - Based on SMILES **rules** (Regex)



SMILES pair encoding

- A SMILES Pair Encoding (SPE) vocabulary is trained with following steps:
 - 1) Tokenize SMILES from a large dataset (e.g., ChEMBL) at atom-level.
 - 2) Initialize the vocabulary with all unique tokens.
 - 3) Iteratively count the occurs of all token pairs in the tokenized SMILES and merge the most frequent occurring token pair as a new token and add it to the vocabulary.
 Stop when:
 - A desired vocabulary size is achieved, or
 - No pair of tokens has frequency larger than the frequency threshold.
- The vocabulary size and frequency threshold are hyperparameters.



SMILES pair encoding

SMILES dataset:

C[C@H](N)C(=O)O
C/C=C/C(=O)O
C[C@H](O)[C@@H](O)C(=O)O
F[C@](Cl)(Br)I
...
....
C[C@H]([C@@H](C(=O)O)N)O
C[C@@H]1CN(C)C[C@H]1C
C[C@H](C)[C@H](C(=O)O)N

Initial set of tokens:

- C
- [
- @
- ...
-]
- (
- N
-)

Additional tokens:

- [C
- [C@
- ...

Special Tokens:

- <bos>
- <eos>
- <pad>
- <mask>
- ...

Most common combination of existing tokens:

1. “[+ “C” = “[C”
2. “[C“ + “@” = “[C@”
3. ...

Regex tokenizer

- Designed by Schwaller et al.
Molecular Transformer: A Model for Uncertainty-Calibrated Chemical Reaction Prediction (2019).
- Uses a vocabulary file (size 591) derived from the USPTO reaction open dataset.

Special Tokens:

- <bos>
- <eos>
- <pad>
- <mask>
- ...

Single Characters:

- (- C
-) - O
- / - N
- \ - F
- - - S
- = - P
- # - I
- 1 - B
- 2 - Cl
- ... - Br
- 9 - c
- o
- ...

Complex Tokens:

- [C@H] - [N-]
- [C@@H] - [Si]
- [C@] - [n+]
- [C@@] - [2H]
- [N+] - [nH]
- [O-] - [Na+]
- [S@] - [Cl-]
- [S@@] - [c-]
- [NH+] - [C-]
- [Fe] - ...
- ...

Positional embeddings

■ Absolute embeddings

- Model can learn which positions are important for the task.
- Might struggle handling sequences longer than the ones seen during training.
- Small parameter overhead.

$$\begin{pmatrix} p_{1,1} & \cdots & p_{1,d} \\ \vdots & \ddots & \vdots \\ p_{512,1} & \cdots & p_{512,d} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} = \begin{pmatrix} p_{2,1} \\ p_{2,2} \\ \vdots \\ p_{2,d} \end{pmatrix}$$

■ Sinusoidal positional encodings

- Lightweight solution that can handle variable-length SMILES.
- May not capture complex positional patterns that are important in chemical graphs.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right),$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

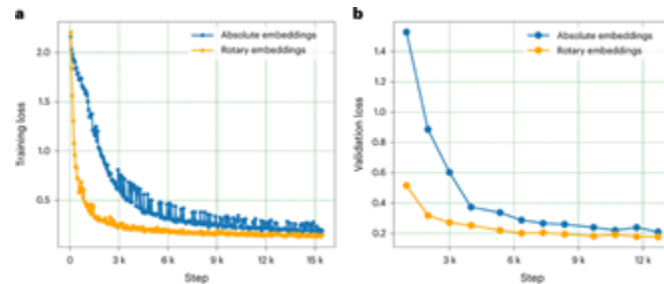
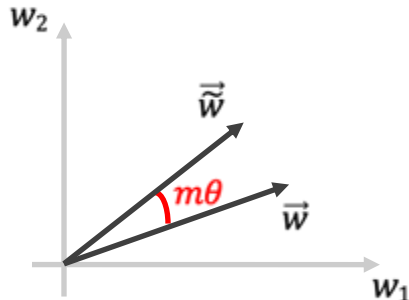
Positional embeddings

■ Rotary Positional Embeddings (RoPE)

- RoPE **rotates** each token's query/key embedding in the attention mechanism by an angle that depends on its absolute position.
- Length extrapolation**: Because rotations are a continuous function of position, you can apply them for positions beyond your training maximum without having to learn new embeddings.
- Chemically meaningful biases**: Relative distances in SMILES often correlate with, ring closures, branch lengths, etc.

m position in input sequence
 θ rotation constant

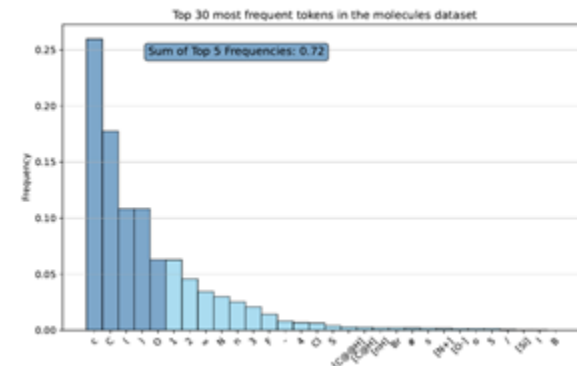
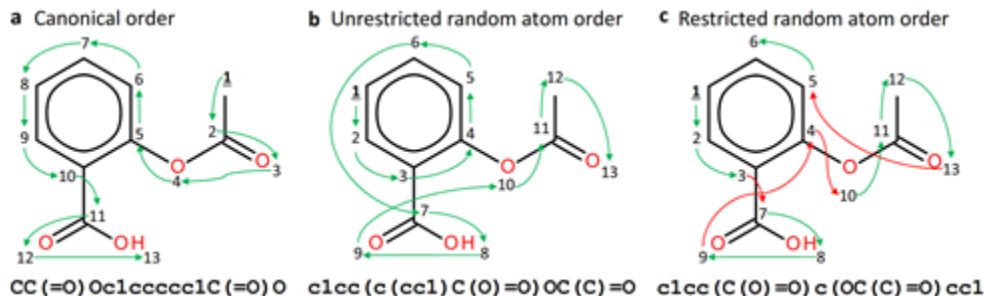
$$\vec{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \xrightarrow{\text{rotate by } m \cdot \theta} \tilde{\vec{w}}$$



Model pretraining

- Masked Language Modelling (MLM):
 - Masking 15% of the tokens in each input string (same as with NLP).
 - Training the model to correctly identify the masked tokens.
 - Problems:
 - In contrast to language/proteins it is much less restricted what token you can have at what positions.
 - Most of the time we are masking carbon atoms.
 - Requires SMILES augmentation to learn better representations.

CC ?



Model pretraining

- Multi-task Regression (MTR)
 - Domain-specific strategy introduced with ChemBERTa-2¹.
 - Compute a set of ~200 molecular properties for each compound in our training dataset.
 - Properties can be calculated from SMILES strings using RDKit.
 - Consistently outperforms MLM on SMILES strings.

MLM vs. MTR

	BACE <i>RMSE</i>	Clearance <i>RMSE</i>	Delaney <i>RMSE</i>	Lipo <i>RMSE</i>	BACE <i>ROC</i>	BBBP <i>ROC</i>	ClinTox <i>ROC</i>	SR-p53 <i>ROC</i>
ChemBERTa-2								
MLM-5M	1.451	54.601	0.946	0.986	0.793	0.701	0.341	0.762
MLM-10M	1.611	53.859	0.961	1.009	0.729	0.696	0.349	0.748
MLM-77M	1.509	52.754	1.025	0.987	0.735	0.698	0.239	0.749
MTR-5M	1.477	50.154	0.874	0.758	0.734	0.742	0.552	0.834
MTR-10M	1.417	48.934	0.858	0.744	0.783	0.733	0.601	0.827
MTR-77M	1.363	48.515	0.889	0.798	0.799	0.728	0.563	0.817

Popular small molecule Transformer

	ChemBERTa-2 ¹	Molformer ²
Architecture	Encoder only	Encoder only
Model size	3.4-43M	~85M
Pre-training task	MTR	MLM
Training set size	5-77M	1.1B
Input	canonical SMILES	canonical SMILES
Pos. embedding	Learned	RoPE
Tokenization	Frequency-based method?	Based on SMILES rules